

Roteiro Prático 07 — Integração Frontend com API no Flask

Objetivos da Aula

Ao final desta aula, o estudante será capaz de:

- Compreender o conceito de **desacoplamento** (separação de Frontend e Backend);
- Transformar uma aplicação Flask em uma **API REST** que responde **JSON**;
- Consumir rotas de API utilizando a **Fetch API** do JavaScript no navegador;
- Manipular a árvore **DOM** para exibir, cadastrar e remover elementos sem recarregar a página.

Ferramentas Necessárias

Python, Flask, JavaScript básico, HTML5 semântico, navegador web, Git.

De onde viemos (Roteiro 05) para onde vamos (Roteiro 07)

Para compreender o salto tecnológico de hoje, vamos analisar as diferenças fundamentais entre o modelo de desenvolvimento dinâmico clássico (**Roteiro 05**) e a arquitetura moderna baseada em APIs (**Roteiro 07**):

Aspecto	Roteiro 05: Flask CRUD (SSR)	Roteiro 07: API e Fetch (CSR)
Papel do Flask	Injeção de Layout + Dados: O Flask processa os dados e monta o HTML no servidor usando Jinja2.	Servidor de Dados Puro: O Flask vira uma API REST que apenas recebe e responde JSON.
Navegação	Recarregamento Completo: Toda ação do usuário (cadastrar/deletar) recarrega a página inteira (tela pisca).	Atualização Cirúrgica: O JavaScript intercepta ações e atualiza cirurgicamente apenas os nós alterados.
Tráfego de Dados	Form Data: Dados via formulário físico do HTML (<code>request.form</code>).	JSON: Dados padronizados e estruturados no corpo da requisição (<code>request.get_json()</code>).
Reuso de Código	Preso ao Layout: Backend acoplado ao HTML/Jinja2. Impossível reutilizar para um app celular.	Totalmente Desacoplado: O mesmo backend atende o site web, apps móveis (iOS/Android) ou outros servidores.

Comparação Prática de Fluxo e Código

1. Rota de Cadastro no Flask

```
# ROTEIRO 05 (Jinja2 / Form Data)
@app.route("/alunos", methods=["POST"])
def adicionar_aluno():
    nome = request.form["nome"]
    idade = int(request.form["idade"])
    alunos.append(Aluno(nome, idade))
    return redirect("/") # Recarrega a tela

# ROTEIRO 07 (API REST / JSON)
@app.route("/api/alunos", methods=["POST"])
def api_adicionar_aluno():
    dados = request.get_json()
    novo_aluno = Aluno(dados["nome"], int(dados["idade"]))
    alunos.append(novo_aluno)
    return jsonify(novo_aluno.to_dict()), 201 # Retorna dados puros
```

2. Exibição de Dados no Frontend

```
<!-- ROTEIRO 05 (Jinja2 no Servidor) -->
<ul>
    {% for aluno in alunos %}
        <li>{{ aluno.nome }} - {{ aluno.idade }} anos</li>
    {% endfor %}
</ul>

<!-- ROTEIRO 07 (DOM Dinâmico no Cliente com JS) -->
<ul id="lista-alunos"></ul>
<script>
    fetch('/api/alunos')
        .then(res => res.json())
        .then(alunos => {
            alunos.forEach(aluno => {
                const li = document.createElement('li');
                li.innerHTML = `<strong>${aluno.nome}</strong> - ${aluno.idade} anos`;
                listaAlunos.appendChild(li);
            });
        });
```



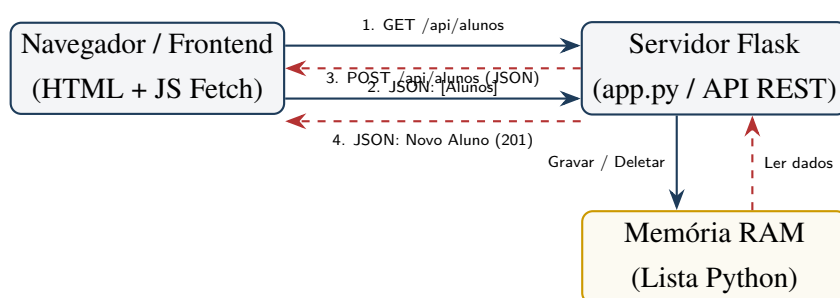
```
});  
</script>
```

Esboço (Croqui) da Interface

Abaixo está o croqui representando como a interface Single Page Application (SPA) será estruturada visualmente. Os dados serão carregados, inseridos e removidos dinamicamente da lista sem recarregar a tela:

Fluxo Geral de Comunicação (REST / SPA)

Abaixo está representado o fluxo de comunicação em segundo plano (assíncrono) entre o cliente (navegador/JavaScript) e o servidor de API (Flask):



Roteiro Prático

Parte 0 – Preparando o Ambiente do Zero



Para garantir que o projeto seja construído do absoluto zero (sem dependências obrigatórias de aulas anteriores), abra o terminal do seu computador e execute os comandos abaixo para criar a pasta, configurar o ambiente virtual isolado e instalar o Flask:

```
# 1. Crie a pasta do projeto e entre nela
mkdir roteiro07-integracao-api
cd roteiro07-integracao-api

# 2. Crie e ative o ambiente virtual (venv)
# No macOS/Linux:
python3 -m venv venv
source venv/bin/activate

# No Windows (CMD):
python -m venv venv
venv\Scripts\activate

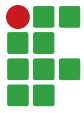
# 3. Instale o Flask e suas dependências
pip install Flask

# 4. Crie a subpasta para as telas HTML
mkdir templates
```

**Parte 1 – Criando o Modelo (classe Aluno)**

Para podermos deletar ou editar alunos específicos através de chamadas de API, precisamos que a classe `Aluno` possua um ID único incremental e consiga exportar seus dados como um dicionário Python (o qual o Flask pode converter em texto bruto JSON).

Crie o arquivo `aluno.py` com a seguinte estrutura completa:



```
class Aluno:
    contador = 1 # Contador estático para gerar IDs únicos

    def __init__(self, nome, idade):
        self.id = Aluno.contador
        self.nome = nome
        self.idade = idade
        self.notas = []
        Aluno.contador += 1

    def adicionar_nota(self, nota):
        self.notas.append(nota)

    def calcular_media(self):
        if not self.notas:
            return 0
        return sum(self.notas) / len(self.notas)

    def aprovado(self):
        return self.calcular_media() >= 7

    # Transforma o objeto em um dicionário compatível com JSON
    def to_dict(self):
        return {
            "id": self.id,
            "nome": self.nome,
            "idade": self.idade,
            "media": self.calcular_media(),
            "aprovado": self.aprovado(),
        }
```

Parte 2 – Construindo a API no Backend (app.py)



No arquivo `app.py`, faremos modificações importantes. A nossa rota principal / apenas entregará a página HTML base uma única vez. Todas as outras interações de dados (listar, cadastrar, deletar) serão feitas por rotas exclusivas de API prefixadas com `/api/`.

Crie ou substitua o conteúdo do arquivo `app.py` com a seguinte estrutura completa:

```
from flask import Flask, jsonify, request, render_template
from aluno import Aluno

app = Flask(__name__)

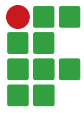
# Banco de dados simulado em memória
alunos = []

# Inicializando com alguns dados de teste
aluno1 = Aluno("João Silva", 17)
aluno1.adicionar_nota(8.5)
aluno2 = Aluno("Maria Souza", 16)
aluno2.adicionar_nota(9.0)

alunos.append(aluno1)
alunos.append(aluno2)

# =====
# 1. ROTA DE TELA (HTML Estático)
# =====
@app.route("/", methods=["GET"])
def pagina_inicial():
    # Retorna o arquivo HTML sem injetar dados via Jinja2
    return render_template("index.html")

# =====
# 2. ROTAS DE API (Endpoints JSON)
# =====
```

```
# ROTA A: Listar Alunos (GET)
@app.route("/api/alunos", methods=["GET"])
def listar_alunos():
    # Converte Alunos para dicionários usando to_dict()
    alunos_dict = [aluno.to_dict() for aluno in alunos]
    return jsonify(alunos_dict), 200

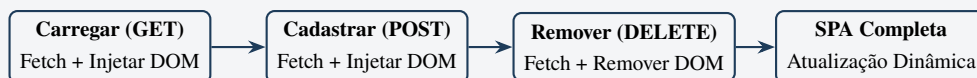
# ROTA B: Adicionar um novo aluno (POST)
@app.route("/api/alunos", methods=["POST"])
def adicionar_aluno():
    # Extrai os dados do corpo da requisição JSON
    dados = request.get_json()
    nome = dados.get("nome")
    idade = dados.get("idade")

    # Valida os dados recebidos
    if not nome or not isinstance(idade, int):
        return jsonify({
            "error": "Dados inválidos. 'nome' deve ser string e 'idade' int."
        }), 400

    # Cria um novo objeto Aluno e adiciona à lista de alunos
    novo_aluno = Aluno(nome, idade)
    alunos.append(novo_aluno)

    return jsonify(novo_aluno.to_dict()), 201

# ROTA C: Remover Aluno (DELETE)
@app.route("/api/alunos/<int:id>", methods=["DELETE"])
def api_remover_aluno(id):
    global alunos
    aluno_existe = any(aluno.id == id for aluno in alunos)
    if not aluno_existe:
        return jsonify({"erro": "Aluno não encontrado"}), 404
    alunos = [aluno for aluno in alunos if aluno.id != id]
    return jsonify({"mensagem": "Aluno removido com sucesso"}), 200
```

**Parte 3 – Criando a Interface no Frontend (index.html)**

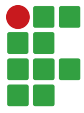
Agora, o nosso arquivo `templates/index.html` é estático e não possui laços do Jinja2. Ele possui um contêiner vazio (`<ul id="lista-alunos"></ul>`) que o JavaScript preencherá dinamicamente usando Fetch API e manipulação do DOM.

Crie o arquivo `templates/index.html` com a estrutura do formulário e o controle assíncrono:

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Controle de Alunos</title>
</head>
<body>
  <h1>Controle de Alunos</h1>
  <form id="formulario-cadastro">
    <input type="text" id="nome" placeholder="Nome do aluno" required>
    <input type="number" id="idade" placeholder="Idade do aluno" required>
    <button type="submit">Cadastrar via API</button>
  </form>
  <hr>
  <h2>Alunos Cadastrados</h2>
  <ul id="lista-alunos"></ul>

  <script>
    const API_URL = '/api/alunos';
    const listaAlunos = document.getElementById('lista-alunos');
    const formulario = document.getElementById('formulario-cadastro');
    const inputNome = document.getElementById('nome');
    const inputIdade = document.getElementById('idade');

    function carregarAlunos() {
      fetch(API_URL)
        .then(response => response.json())
```



```
.then(alunos => {
    listaAlunos.innerHTML = '';
    alunos.forEach(aluno => {
        desenharAlunoNaTela(aluno);
    });
});

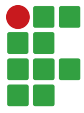
}

// Carrega os alunos colocando-os como novos
// elementos em uma lista HTML
function desenharAlunoNaTela(aluno) {
    const li = document.createElement('li');
    li.id = `aluno-${aluno.id}`;
    li.innerHTML = `
        <span><strong>${aluno.nome}</strong>
            (${aluno.idade} anos)</span>
        <button onclick="deletarAluno(${aluno.id})">
            Deletar
        </button>
    `;
    listaAlunos.appendChild(li);
}

formulario.addEventListener('submit', function (event) {
    // Evita o envio tradicional/recarga padrão
    event.preventDefault();

    const dados = {
        nome: inputNome.value,
        idade: parseInt(inputIdade.value)
    };

    fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dados) // Envia como JSON
    })
```



```
.then(res => res.json()) // Converte em JSON
.then(aluno => {
    desenharAlunoNaTela(aluno); // Desenha aluno
    formulario.reset(); // Limpa campos
});

});

function deletarAluno(id) {
    if (!confirm('Deseja realmente deletar?')) return;

    fetch(`${API_URL}/${id}`, { method: 'DELETE' })
        .then(response => {
            if (response.ok) {
                const el = document.getElementById(
                    `aluno-${id}`
                );
                if (el) el.remove(); // Remove da tela
            } else {
                alert('Erro ao deletar o aluno.');
```

Testando a Aplicação de Forma Segura

Siga as etapas abaixo para executar e inspecionar sua aplicação, diferenciando a dinâmica desta arquitetura daquela desenvolvida no **Roteiro 05**:

1. No terminal com o ambiente virtual ativo, inicie o servidor:

```
python app.py
```

2. Abra o navegador no endereço: `http://localhost:5001` (ou na porta configurada no seu script).
3. Abra a Ferramenta do Desenvolvedor (**F12** ou clique com o botão direito → *Inspecionar*).
4. Vá na aba **Rede (Network)** e selecione o filtro **Fetch/XHR**.
5. Cadastre um aluno ou delete um registro existente e observe a diferença:
 - **No Roteiro 05 (SSR):** O formulário enviava uma requisição física, o servidor gerava uma nova página HTML inteira e forçava o navegador a dar um *hard reload* (a tela piscava em branco e limpava o log da aba Rede).
 - **No Roteiro 07 (API/SPA):** A página permanece estática e **não recarrega** (não pisca). Na aba **Fetch/XHR**, você verá surgir conexões assíncronas em segundo plano: um POST para `/api/alunos` ao cadastrar e um DELETE para `/api/alunos/<id>` ao remover.
6. Dê um duplo clique em qualquer uma dessas requisições de Fetch na aba Rede para inspecionar os dados puros no formato **JSON** recebidos e enviados pelo servidor.

A Figura abaixo demonstra o resultado final da aplicação Single Page Application (SPA) estilizada funcionando no navegador:

Resultado Final — Interface SPA Estilizada

Controle de Alunos

Cadastrar via API

Alunos Cadastrados

- Alice (20 anos) Deletar
- André Moraes (44 anos) Deletar
- Priscila de Oliveira (22 anos) Deletar

A Figura abaixo mostra a aba **Rede (Network)** do DevTools com o filtro **Fetch/XHR** ativo, evidenciando as requisições assíncronas realizadas pela aplicação em segundo plano — sem recarregar a página:

Requisições Assíncronas — Filtro Fetch/XHR no DevTools

Nome	Domínio	Tipo	Iniciador	Tam. ...	Hora
alunos	localhost	obter	localhost:63	270 KB	25,8ms
alunos	localhost	obter	localhost:63	272 KB	20,5ms
alunos	localhost	obter	localhost:63	283 ...	10,9ms
alunos	localhost	obter	localhost:63	273 KB	10,9ms
1	localhost	obter	localhost:79	222 KB	13,1ms

Estilização da Interface (Opcional — CSS Premium)

Para dar um acabamento profissional e moderno à interface da sua aplicação Single Page Application (SPA), você pode criar uma folha de estilos personalizada.

Siga os passos abaixo para aplicar a estilização:

1. Na raiz do seu projeto, crie uma pasta para arquivos estáticos e crie o arquivo CSS:

```
mkdir static
# Crie o arquivo static/style.css
```

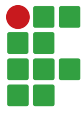
2. Adicione as seguintes regras CSS premium ao arquivo static/style.css:

```
@import url(
  'https://fonts.googleapis.com/css2?family=Inter:wght@400;700'
);

:root {
  --bg-color: #f4f7f6;
  --card-bg: #ffffff;
  --primary-color: #1e3c5a;
  --primary-hover: #122538;
  --text-color: #333333;
  --border-color: #e2e8f0;
  --danger-color: #b43232;
  --danger-hover: #8c2525;
}

body {
  font-family: 'Inter', sans-serif;
  background-color: var(--bg-color);
  color: var(--text-color);
  max-width: 600px;
  margin: 40px auto;
  padding: 20px;
}

h1, h2 {
  color: var(--primary-color);
  font-weight: 700;
}
```



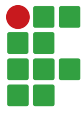
```
form {
  background-color: var(--card-bg);
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.1);
  margin-bottom: 30px;
  display: flex;
  flex-direction: column;
  gap: 15px;
}

input {
  padding: 12px;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  font-size: 1em;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}

input:focus {
  border-color: var(--primary-color);
  box-shadow: 0 0 0 3px rgba(30,60,90,0.15);
}

button[type="submit"] {
  padding: 12px;
  background-color: var(--primary-color);
  color: white;
  font-weight: 600;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background-color 0.2s, transform 0.1s;
}

button[type="submit"]:hover {
```

```
        background-color: var(--primary-hover);
    }

    button[type="submit"]:active {
        transform: scale(0.98);
    }

    ul {
        list-style: none;
        padding: 0;
        display: flex;
        flex-direction: column;
        gap: 10px;
    }

    li {
        background-color: var(--card-bg);
        padding: 16px;
        border-radius: 8px;
        box-shadow: 0 1px 3px rgba(0,0,0,0.05);
        display: flex;
        justify-content: space-between;
        align-items: center;
        border: 1px solid var(--border-color);
        transition: transform 0.2s;
    }

    li:hover {
        transform: translateY(-2px);
    }

    .btn-deletar {
        background-color: var(--danger-color);
        color: white;
        padding: 8px 16px;
        border: none;
        border-radius: 6px;
    }
```



```
font-size: 0.9em;
font-weight: 600;
cursor: pointer;
transition: background-color 0.2s;
}

.btn-deletar:hover {
    background-color: var(--danger-hover);
}
```

3. No arquivo `templates/index.html`, adicione o link para o CSS dentro da tag `<head>`:

```
<link rel="stylesheet" href="/static/style.css">
```

4. Para que o botão de deletar adote o estilo correto, mude sua classe no script do `index.html`:

```
<button class="btn-deletar"
        onclick="deletarAluno(${aluno.id})">
    Deletar
</button>
```

Entrega

O estudante deverá demonstrar ao professor:

- A API respondendo a lista de alunos em formato JSON via endpoint GET;
- A inserção dinâmica de novos alunos sem recarregar a tela (POST);
- A remoção assíncrona do elemento HTML correspondente ao clicar em "Remover"(DELETE).